

Guía de desarrollo

Censo Municipal de Animales · Navalcarnero

Guía de desarrollo

Requisitos

- Python 3.10+
- MariaDB/MySQL local (127.0.0.1:3306, root/123 por defecto).
- Editor con soporte de Python y JS.

Setup rápido

```
python -m venv .env
. .env/Scripts/activate          # Windows
# . .env/bin/activate            # Linux/macOS
pip install -r api/requirements.txt -r api/requirements-dev.txt
```

```
# Inicializar BD (igual que en docs/DESPLIEGUE.md)
python init_db_local.py
python run_local.py
```

Hot-reload del backend: arranca Flask manualmente con `FLASK_DEBUG=1`.

```
cd api && FLASK_DEBUG=1 python app.py
```

Layout del repositorio

api/	Backend Flask (app.py es monolítico a propósito)
web/	Frontend (HTML+CSS+JS vanilla, sin framework ni build)
db/	Esquema, migraciones y carpeta de backups
tests/	Pytest (mocking de mysql.connector)
scripts/	Utilidades varias
docs/	Esta documentación

Estilo

Backend (Python)

- PEP 8, líneas ~100 columnas.
- Tipado opcional pero bienvenido en helpers nuevos.
- Logging vía `logger` global (configurado al arrancar).
- Toda acción relevante debe llamar a `_log_auditoria(accion, payload, exito=True/False)`.
- Validaciones con helpers `_validar_nif`, `_validar_chip`,

`_validar_incidencia`. Reutiliza, no copies.

Frontend

- Vanilla JS, sin bundler. Cada vista en su propio fichero.
- Convención: una función pública por flujo (ej. `submitAnimal`,

`cargarAnimales`, `abrirFicha`). Sin clases.

- Mensajes de error al usuario vía `mostrarAlerta(idAlert, "ok"|"error", msg)`.

- Errores capturados deben llamar a `logError(message, "fuente", stack)` para

que aparezcan en `/api/log`.

- Estilo BEM-ligero en CSS (`.bloque`, `.bloque-titulo`, `.form-grid`, ...).

Tests

```
python -m pytest # toda la suite
python -m pytest tests/test_validadores.py -v
python -m pytest -k "incidencias"
```

`tests/conftest.py` hace stub de `mysql.connector` para que no necesite BD real. Si añades un endpoint que toca tablas nuevas, añade su test correspondiente.

Añadir un campo a una tabla existente

1. Crea `db/migrations/NN_descripcion.sql` con un `ALTER TABLE` idempotente (verifica si la columna ya existe, o usa `MODIFY` cuando aplique). 2. Si afecta a creación de tablas auto-inicializadas en `app.py` (p.ej. `_init_incidencias`), añade el `ADD COLUMN` envuelto en `try/except` con tolerancia al error 1060. 3. Actualiza `docs/BASE_DE_DATOS.md` con la nueva fila. 4. Añade la fila al `CHANGELOG.md`.

Añadir un endpoint nuevo

1. Define la ruta en `api/app.py` siguiendo el patrón de las existentes:

- Validación temprana → conexión → operación → `_log_auditoria` →

respuesta JSON.

- Manejo de `mysql.connector.IntegrityError` y `Exception` con

`_log_request_error`. 2. Si requiere admin, llama a `_req_admin()`; si requiere policía, `_req_policia_o_admin()`. 3. Añade un test en `tests/test_<dominio>.py`. 4. Documenta la ruta en `docs/API.md`.

Subida de archivos

Los helpers `_guardar_foto_animal` y `_guardar_foto_incidencia` validan extensión, sanean el nombre y guardan en su carpeta. Si añades un nuevo tipo de subida:

- Reutiliza el set `_FOTO_EXT_PERMITIDAS`.
- Persiste **siempre la ruta relativa** servida por el frontend

(`/<carpeta>/<archivo>`), no la ruta absoluta del filesystem.

- La carpeta destino debe vivir bajo `web/` para que la sirva nginx /

`run_local.py` como estática.

Cosas que NO hacer

- No metas pip-deps para tareas triviales (datetime parsing, etc.).

- No uses ORM: el proyecto usa `mysql.connector` con SQL plano y

parámetros (%s).

- No expongas endpoints administrativos sin `_req_admin()`.
- No guardes contraseñas en claro (siempre `bcrypt`).
- No metas dependencias JS externas en `web/` salvo `Chart.js` (vía CDN).

Build / release

No hay paso de build. El despliegue es: 1. `git pull`. 2. `docker compose build --no-cache && docker compose up -d`. 3. Aplicar migraciones nuevas si las hay. 4. Verificar `docker compose logs -f api`.